Reviewer Pack — Minimal Rank of Noncrossing Partitions in Algebraic Combinat…

Entry d30c82a84664 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 12:04:28 UTC

Minimal Rank of Noncrossing Partitions in Algebraic Combinatorics vs Permutation Circuit Threshold

Entry ID: `d30c82a84664` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 12:04:28 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics **Field B** (complexity object): Complexity Theory: Permutation Circuit Complexity

Statement:

> ['For any permutation φ with n elements, the minimal rank of its associated noncrossing partition is Ω(n^(2/3)) and O(φ(n) log^2 n), where φ(n) is the number of inversions in φ.', 'Furthermore, for all permutations with at most n elements, there exists a permutation circuit of linear depth that computes φ such that its size is Θ(min(rank(perm_n), φ(n) log^2 n)).']

Rationale (proposer's reasoning):

> ['Algebraic combinatorics provides tools to study structures like noncrossing partitions, which could yield insights into the complexity of computational problems.', "The invariant's rank serves as a measure that is likely to grow with the size of the permutation, potentially offering a threshold for circuit complexity."]

Taxonomy category: `GCT_DET_PERM` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `24767f0f5bf9d3d8`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for at least 80% of random permutations φ, the minimal rank of the associated noncrossing partition is at least Ω(n^(2/3)) AND at most O(φ(n) log^2 n), where φ(n) is the number of inversions in φ. The conjecture is falsified if any seed produces a permutation with a minimal rank below Ω(n^(2/3)) or above O(φ(n) log^2 n).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebraic combinatorics AND minimal rank noncrossing partitions - permutation circuit threshold AND inversions in permutations - noncrossing partition minimal rank AND permutation circuit size

Top relevant hits considered: - [http://arxiv.org/abs/2212.13799v6] Noncrossing partitions of a marked surface - [http://arxiv.org/abs/1604.06009v2] Oriented Flip Graphs and Noncrossing Tree Partitions - [http://arxiv.org/abs/1904.05573v3] k-Indivisible Noncrossing Partitions - [http://arxiv.org/abs/2312.01182v2] Thresholds for patterns in random permutations with a given number of inversions - [http://arxiv.org/abs/1908.07277v2] Permutations with few inversions are locally uniform - [http://arxiv.org/abs/1512.02171v5] Right-jumps and pattern avoiding permutations - [http://arxiv.org/abs/1509.06942v2] Symmetric Decompositions and the Strong Sperner Property for Noncrossing Partition Lattices

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_permutation(n):
        return list(range(1, n + 1))

    def inversions_count(perm):
        count = 0
        for i in range(len(perm)):
            for j in range(i + 1, len(perm)):
                if perm[i] > perm[j]:
                    count += 1
        return count

    def noncrossing_partition_size(n):
        # Using a simple heuristic to estimate the size of the partition
        return math.ceil(n ** (2/3))
```

```
    n = random.randint(5, 40)
    perm = generate_permutation(n)
    inv_count = inversions_count(perm)
    rank = noncrossing_partition_size(n)

    # Construct a permutation circuit (simplified version)
    circuit_depth = max(1, math.ceil(math.log2(rank)))
    circuit_size = rank * circuit_depth

    metric_value = rank
    instances_tested = 1
    conjecture_holds = Ω_n_2_3 <= rank <= O_inv_log2_n
    counterexample = "" if conjecture_holds else f"Rank {rank} does not satisfy the bounds for n={

    return {
        "metric_name": "rank",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys

    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = random.sample(primes, min(30, len(primes)))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_rank) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_dev} support_fraction={support_fractio
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"Rank does not satisfy the bounds\" first_failin
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ee043afc.py", line 69, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ee043afc.py", line 47, in run_trial
    conjecture_holds = Ω_n_2_3 <= rank <= O_inv_log2_n
                       ^^^^^^^
NameError: name 'Ω_n_2_3' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing any data, which means we cannot verify if the conjecture is supported or falsified according to the pre-registered support and falsification conditions. | next: Re-run the test without errors to collect data for verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14119
2	preregistration	ollama_remote	glm4:latest	0	0	16441
3	novelty	ollama_remote	glm4:latest	0	0	8240
4	novelty	ollama_remote	glm4:latest	0	0	9548
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17045
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10348
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7462
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9180
9	judge	ollama_remote	glm4:latest	0	0	14164

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 106547 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d30c82a84664.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d30c82a84664.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d30c82a84664.tar.gz` (if generated)